

Intelligenza Artificiale e Laboratorio

Planning, Sistemi a Regole e Incertezza

Roberto Micalizio

a.a. 2025/2026

Università degli Studi di Torino,
Dipartimento di Informatica



Planning nello Spazio degli Stati

Classical Planning Problem

Un problema di pianificazione classica è $\mathcal{P} = (\Sigma, s_0, S_g)$

- $\Sigma = (S, A, \gamma)$ è il modello del dominio espresso come STS
- $s_0 \in S$ è lo stato iniziale
- $S_g \subset S$ è l'insieme degli stati goal

Una soluzione π ad un problema $\mathcal{P}$

- una sequenza totalmente ordinata di azioni istanziate (**ground**)
 $\pi = \langle a_1, a_2, \dots, a_n \rangle$
- che danno origine ad una sequenza di transizioni di stato
 $\langle s_0, s_1, \dots, s_n \rangle$ tale che:
 - $s_1 = \gamma(s_0, a_1)$
 - $\forall k : 2..n \ s_k = \gamma(s_{k-1}, a_k)$
 - $s_n \in S_g$

- In prima approssimazione: **Planning come algoritmi di ricerca** (es. BFS, DFS, A*, ecc.)
- **MA** esistono delle differenze:
 - fattorizzazione del modello del mondo in termini di **schemi di azioni**
→ lo stato del sistema non è atomico, ma costituito da proposizioni
 - **spazio di ricerca** $\neq$ **spazio degli stati**
 - lo spazio degli stati è indotto dal transition system Σ
 - lo spazio di ricerca dipende dall'algoritmo di pianificazione
(non sempre un algoritmo cerca nello spazio degli stati, vedi least-commitment planning)

Algoritmi di pianificazione: Molte scelte da fare

1. Direzione della ricerca

- **Progression**: in avanti dallo stato iniziale al goal
- **Regression**: dal goal allo stato iniziale
- **Bidirezionale**

2. Rappresentazione dello spazio di ricerca

- spazio di ricerca **esplicito**: coincide con lo spazio degli stati del transition system che modella il dominio
- spazio di ricerca **simbolico** in cui ogni stato corrisponde a insiemi di stati del dominio

3. Algoritmo di ricerca

- **Non informato:** depth-first, breadth-first, iterative deepening, ecc.
- **Ricerca euristica sistematica:** greedy best-first, A^* , IDA*, ecc.
- **Ricerca euristica locale:** hill-climbing, simulated annealing, beam search, ecc.

4. Controllo della ricerca

- euristiche per gli algoritmi informati
- pruning techniques: partial-order reduction, helpful actions pruning, symmetry elimination, ecc.

FF (Hoffmann & Nebel, 2001)

- Ricerca in avanti
- Spazio di ricerca esplicito
- Algoritmo di ricerca: hill-climbing
- Euristica inammissibile + pruning delle azioni
- Subottimo

LAMA (Richter & Westphal, 2008)

- Ricerca in avanti
- Spazio di ricerca esplicito
- Algoritmo di ricerca: variante di A^*
- Euristica FF, landmark (non ammissibile)
- Subottimo

Fast Downward (Helmert et al., 2011)

- Ricerca in avanti
- Spazio di ricerca esplicito
- Algoritmo di ricerca: A*
- LM-cut; merge-and-shrink; landmarks; (ammissibili)
- Ottimo

Progression

Fattorizzazione in termini di schemi di azioni

- Schema di azione o *plan operator* modella un'azione in termini generici, è un *template* da cui è possibile istanziare più azioni ground (prive di variabili)
- un plan operator o stabilisce:
 - i parametri
 - le precondizioni $pre(o)$ come vincoli sui parametri
 - gli effetti $effects(o)$ come modifiche che saranno riportate ai parametri
- In genere il linguaggio usato è quello della logica proposizionale
 - precondizioni ed effetti possono essere ricondotte ad operazioni insiemistiche
 - $effects^+(o)$ effetti positivi: aggiunta di proposizioni
 - $effects^-(o)$ effetti negativi: rimozione di proposizioni
 - saremo più precisi quando parleremo di STRIPS

Progression: Ricerca in Avanti

- **Progression**: ricerca per **azioni applicabili**
- calcolo dello stato successore s' di uno stato s rispetto all'applicazione di un operatore o :

$$s' = \gamma(s, o)$$

- Pianificatori basati su progression applicano delle ricerche in avanti tipicamente nello spazio degli stati:
 - la ricerca comincia da uno stato iniziale
 - iterativamente, viene selezionato uno stato s precedentemente generato al quale viene applicato un operatore o per generare un nuovo stato s'
 - la soluzione è trovata quando lo stato s' appena generato soddisfa il goal ($s' \models s_g$ e $s_g \in S_g$).
- ha il vantaggio di essere molto intuitivo e facile da implementare

Progression: Strategia

```
function fwdSearch( $O, s_0, s_g$ )  
   $state \leftarrow s_0$   
   $\pi \leftarrow \langle \rangle$   
  loop  
    if  $state.satisfies(s_g)$  then return  $\pi$   
     $applicables \leftarrow \{\text{istanze ground da } O \text{ applicabili in } state\}$   
    if  $applicables.isEmpty()$  then return failure  
     $action \leftarrow applicables.chooseOne()$   
     $state \leftarrow \gamma(state, action)$   
     $\pi \leftarrow \pi \circ \langle action \rangle$   
  return failure
```

NB. $applicables.chooseOne()$ rappresenta una scelta **non deterministica**, ma **esaustiva**, di un'azione

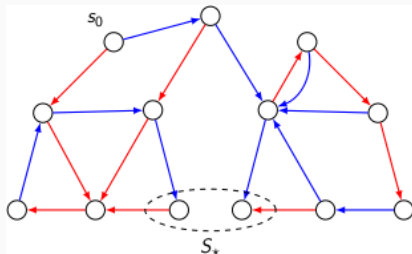
Soundness

`fwdSearch` è corretto: se la funzione termina con un piano come soluzione, allora questo piano è effettivamente una soluzione al problema iniziale

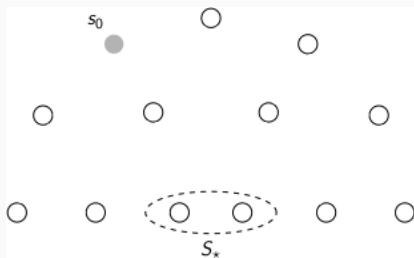
Completeness

`fwdSearch` è completo: se esiste una soluzione allora esiste una traccia d'esecuzione che restituirà quella soluzione come piano

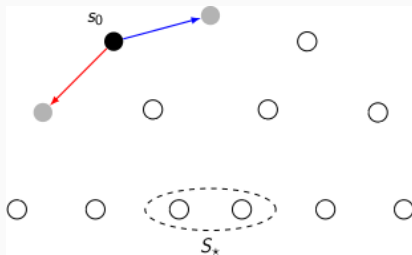
Esempio di forward search



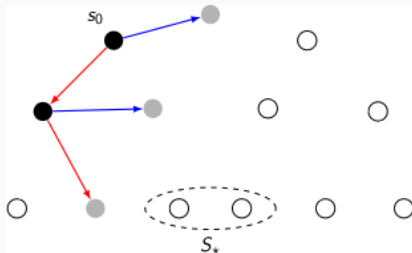
Esempio di forward search



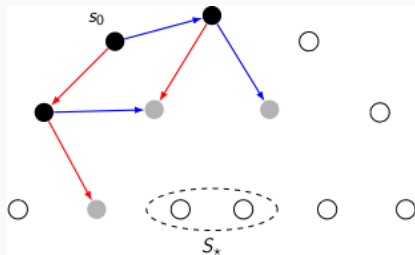
Esempio di forward search



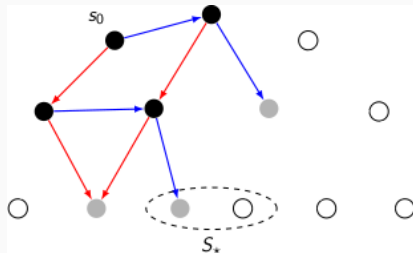
Esempio di forward search



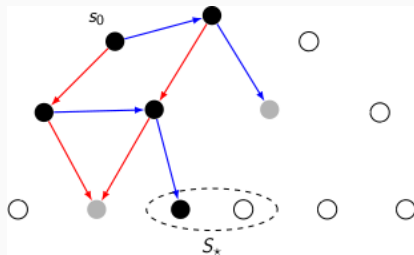
Esempio di forward search



Esempio di forward search



Esempio di forward search



Regression

Problemi della Ricerca in Avanti

- il numero di azioni applicabili in un dato stato è in genere molto grande
- di conseguenza anche il *branching factor* tende ad essere grande
- la ricerca in avanti corre il rischio di non essere praticabile dopo pochi passi
- **Regression**: ricerca all'indietro partendo dal goal
- **MA** occorre saper trattare insiemi di stati goal, cioè belief states

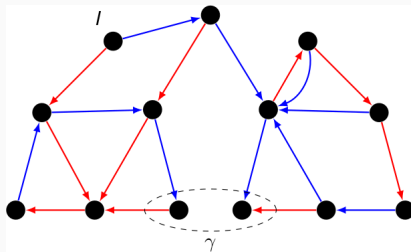
- **Regression**: ricerca per **azioni rilevanti**
- Ricerca all'indietro:
 - comincia dall'insieme di stati goal
 - iterativamente, seleziona un sottogoal generato precedentemente, e “regrediscilo” attraverso un operatore, generando così un nuovo sottogoal
 - la soluzione è trovata quando il nuovo sottogoal è soddisfatto dallo stato iniziale
- ha il vantaggio di poter gestire più stati simultaneamente — > **belief states** (quindi lo spazio di ricerca esplorato è più compatto)
- gestire belief state è anche più costoso e difficile

Regression

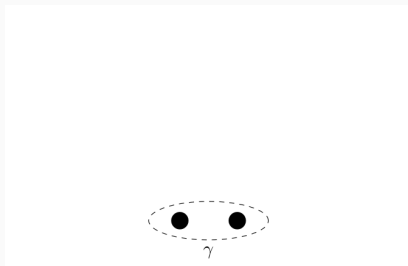
- Il calcolo del **regresso**, ovvero il *sottogoal predecessore di un goal dato*, avviene nel seguente modo:
 - dato un goal g
 - sia a azione ground tale che $g \in effects^+(a)$
 - $g' = \gamma^{-1}(g, a) = (g \setminus effects^+(a)) \cup pre(a)$
- g' è quindi il regresso di g attraverso la relazione di transizione γ e l'azione a
- NB. *nel calcolo del predecessore non sono considerati gli effetti negativi perché?*
- come vedremo STRIPS introduce la nozione di plan operator (alla base di PDDL) proprio per implementare la regressione

```
function bwdSearch( $O, s_0, g$ )  
   $subgoal \leftarrow g$   
   $\pi \leftarrow \langle \rangle$   
  loop  
    if  $s_0.satisfies(subgoal)$  then return  $\pi$   
     $relevants \leftarrow \{\text{ground instances from } O \text{ relevant for subgoal}\}$   
    if  $relevants.isEmpty()$  then return failure  
     $action \leftarrow relevants.chooseOne()$   
     $subgoal \leftarrow \gamma^{-1}(subgoal, action)$   
     $\pi \leftarrow \langle action \rangle \circ \pi$ 
```

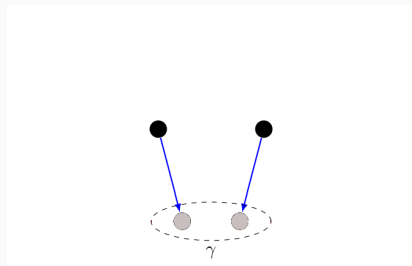
Esempio di backward search



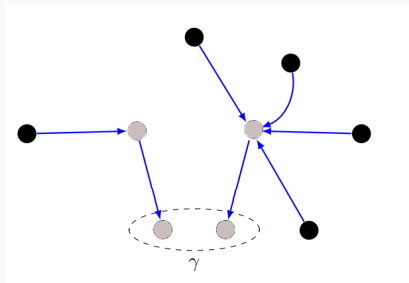
Esempio di backward search



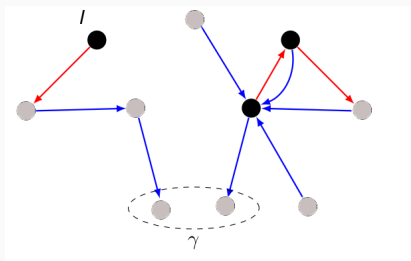
Esempio di backward search



Esempio di backward search



Esempio di backward search



Progression vs. Regression

- Ricerca in avanti comincia da un singolo stato iniziale
- Quando si applica in avanti un operatore o ad uno stato s si genera un unico stato successore s'
- Nella ricerca in avanti lo spazio di ricerca coincide con lo spazio degli stati (stati del dominio originati da Σ) (in genere)

Progression vs. Regression

- Ricerca in avanti comincia da un singolo stato iniziale
Ricerca all'indietro comincia da un insieme di stati goal
- Quando si applica in avanti un operatore o ad uno stato s si genera un unico stato successore s'
- Nella ricerca in avanti lo spazio di ricerca coincide con lo spazio degli stati (stati del dominio originati da Σ) (in genere)

Progression vs. Regression

- Ricerca in avanti comincia da un singolo stato iniziale
Ricerca all'indietro comincia da un insieme di stati goal
- Quando si applica in avanti un operatore o ad uno stato s si genera un unico stato successore s'
Quando da uno stato s' vogliamo fare un passo all'indietro scopriamo che possono esserci molteplici stati predecessori
- Nella ricerca in avanti lo spazio di ricerca coincide con lo spazio degli stati (stati del dominio originati da Σ) (in genere)

Progression vs. Regression

- Ricerca in avanti comincia da un singolo stato iniziale
Ricerca all'indietro comincia da un insieme di stati goal
- Quando si applica in avanti un operatore o ad uno stato s si genera un unico stato successore s'
Quando da uno stato s' vogliamo fare un passo all'indietro scopriamo che possono esserci molteplici stati predecessori
- Nella ricerca in avanti lo spazio di ricerca coincide con lo spazio degli stati (stati del dominio originati da Σ) (in genere)
Nella ricerca all'indietro ogni stato dello spazio di ricerca corrisponde ad un insieme di stati del dominio

Regression: Note conclusive

- La ricerca all'indietro si basa su azioni **rilevanti**, cioè quelle che contribuiscono attivamente al goal perché
 - producono almeno uno degli atomi che compaiono nel goal e
 - non hanno effetti negativi sul goal stesso, cioè che non negano uno degli atomi del goal
- La ricerca all'indietro mantiene un fattore di ramificazione più basso rispetto alla ricerca in avanti, ma il fatto di dover mantenere un belief state può complicare le strutture dati usate dal planner e la definizione di euristiche
- nonostante il vantaggio teorico, la ricerca in avanti è preferita alla ricerca all'indietro

STRIPS

- **STanford Research Institute Problem Solver** [Fikes, Nilsson, 1971]
 - Introduce una rappresentazione esplicita degli **operatori di pianificazione**
 - Fornisce una operazionalizzazione delle nozioni di
 - differenza tra stati
 - subgoal
 - applicazione di un operatore
 - Gestisce il Frame Problem
 - Si basa su due idee fondamentali:
 - Linear Planning
 - Means-End Analysis

- Idea di base:
 - Risolvere un goal per volta, e passare al successivo solo quando il precedente è stato raggiunto
- L'algoritmo di planning mantiene uno **Stack dei Goal** (risolvere un goal può comportare la risoluzione di sotto-goal)
- Conseguenze:
 - Non c'è interleaving nel conseguimento dei goal
 - La ricerca è efficiente se i goal non interferiscono troppo tra loro
 - Ma è soggetto alla **Sussmann's Anomaly**

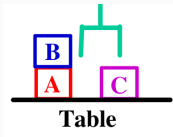
- Idea di base:
 - Considera solamente gli aspetti rilevanti al problema (backward)
 - Quali mezzi (*means*, cioè operatori) sono disponibili e necessari per raggiungere il goal (*end*)
- Occorre stabilire quali differenze ci sono tra lo stato corrente e il goal (means-ends analysis)
- Trovare un operatore che riduce tale differenza
- Ripetere l'analisi sui sottogoal ottenuti per regressione attraverso l'operatore selezionato

Stati in STRIPS

- STRIPS Language $\subset$ FOL¹
 - numero finito di simboli di predicati, simboli di costanti e **senza** simboli di funzione né quantificatori
- Uno stato in STRIPS è una congiunzione di **atomi ground** (privi di simboli di funzione)
- Vale l'assunzione di mondo chiuso (**closed world assumption**)
- Semantica
 - un atomo ground p vale in uno stato s , denotato come $s \models p$ se e solo se $p \in s$
 - uno stato s soddisfa una congiunzione di letterali (atomi con segno) $g_1, g_2, \dots, g_n$, denotato come $s \models g_1, g_2, \dots, g_n$ se:
 - ogni letterale positivo in g_i occorre in s : $g_i \in s$
 - ogni letterale negativo in g_i non occorre in s : $g_i \notin s$

¹First-Order Logic

Esempio: Blocks world



$s = \{ \text{On}(B, A),$
 $\text{OnTable}(C),$
 $\text{OnTable}(A),$
 $\text{Block}(A),$
 $\text{Block}(B),$
 $\text{Block}(C),$
 $\text{Handempty} \}$

- **Relazioni Fluenti:**

- Predicati che rappresentano relazioni il cui valore di verità può cambiare da uno stato al successivo
- Tali predicati sono anche detti **fluente**
- esempio: On; OnTable; ...

- **Relazioni Persistenti** (o state invariant)

- Predicati il cui valore di verità non può cambiare
- esempio: Block() è una relazione (unaria) persistente perché non può cambiare per effetto di una qualche azione

Plan Operators in STRIPS

- Un operatore di pianificazione in STRIPS è una tripla:

$o : (name(o), precondition(o), effects(o))$ dove:

- $name(o)$ è una espressione sintattica con forma

$$n(x_1, \dots, x_k)$$

dove n è il nome dell'operatore, e $x_1, \dots, x_k$ è una lista variabili che compaiono in o

- $precond(o)$ è l'insieme di predicati che rappresentano le precondizioni dell'operatore; i predicati saranno poi istanziati in letterali nelle azioni

- $precond^+(o)$ denota i letterali positivi
- $precond^-(o)$ denota i letterali negativi

- $effects(o)$ è l'insieme di predicati che rappresentano gli effetti dell'operatore:

una variabile può comparire negli effetti solo se è anche menzionata nelle precondizioni (variabili bounded)

- $effects^+(o)$ denota i letterali positivi (add-list)
- $effects^-(o)$ denota i letterali negativi (delete-list)

Plan Operators in STRIPS

- Un operatore di pianificazione in STRIPS è una tripla:

$o : (name(o), precondition(o), effects(o))$ dove:

- $name(o)$ è una espressione sintattica con forma

$$n(x_1, \dots, x_k)$$

dove n è il nome dell'operatore, e $x_1, \dots, x_k$ è una lista variabili che compaiono in o

- $precond(o)$ è l'insieme di predicati che rappresentano le precondizioni dell'operatore; i predicati saranno poi istanziati in letterali nelle azioni

- $precond^+(o)$ denota i letterali positivi

- $precond^-(o)$ denota i letterali negativi

- $effects(o)$ è l'insieme di predicati che rappresentano gli effetti dell'operatore:

una variabile può comparire negli effetti solo se è anche menzionata nelle precondizioni (variabili bounded)

- $effects^+(o)$ denota i letterali positivi (add-list)

- $effects^-(o)$ denota i letterali negativi (delete-list)

Plan Operators in STRIPS

- Un operatore di pianificazione in STRIPS è una tripla:

$o : (name(o), precondition(o), effects(o))$ dove:

- $name(o)$ è una espressione sintattica con forma

$$n(x_1, \dots, x_k)$$

dove n è il nome dell'operatore, e $x_1, \dots, x_k$ è una lista variabili che compaiono in o

- $precond(o)$ è l'insieme di predicati che rappresentano le precondizioni dell'operatore; i predicati saranno poi istanziati in letterali nelle azioni

- $precond^+(o)$ denota i letterali positivi
- $precond^-(o)$ denota i letterali negativi

- $effects(o)$ è l'insieme di predicati che rappresentano gli effetti dell'operatore:

una variabile può comparire negli effetti solo se è anche menzionata nelle precondizioni (variabili bounded)

- $effects^+(o)$ denota i letterali positivi (add-list)
- $effects^-(o)$ denota i letterali negativi (delete-list)

Plan Operators in STRIPS

- Un operatore di pianificazione in STRIPS è una tripla:

$o : (name(o), precondition(o), effects(o))$ dove:

- $name(o)$ è una espressione sintattica con forma

$$n(x_1, \dots, x_k)$$

dove n è il nome dell'operatore, e $x_1, \dots, x_k$ è una lista variabili che compaiono in o

- $precond(o)$ è l'insieme di predicati che rappresentano le precondizioni dell'operatore; i predicati saranno poi istanziati in letterali nelle azioni

- $precond^+(o)$ denota i letterali positivi
- $precond^-(o)$ denota i letterali negativi

- $effects(o)$ è l'insieme di predicati che rappresentano gli effetti dell'operatore:

una variabile può comparire negli effetti solo se è anche menzionata nelle precondizioni (variabili bounded)

- $effects^+(o)$ denota i letterali positivi (add-list)
- $effects^-(o)$ denota i letterali negativi (delete-list)

Esempio: BW Operators

Pick_Block(?b, ?c)

Pre: Block(?b), Handempty, Clear(?b), On(?b, ?c), Block(?c)

Eff: Holding(?b), Clear(?c), $\neg$ Handempty, $\neg$ On(?b, ?c)

Put_Block(?b, ?c)

Pre: Block(?b), Holding(?b), Clear(?c), Block(?c)

Eff: $\neg$ Holding(?b), $\neg$ Clear(?c), Handempty, On(?b, ?c)

NB. Questa modellazione è molto semplificata e incompleta, cosa manca?
un'azione in STRIPS è una istanziazione ground di un plan operator

Progression

- Sia s uno stato del mondo ($s \in S$ per un dato dominio Σ)
 - Sia a un'azione (istanziamento di un plan operator o)
 - Diremo che a è **applicabile** in s se e solo se:
 - $precond^+(a) \subseteq s$
 - $precond^-(a) \cap s = \emptyset$
 - la funzione di transizione di stato $\gamma(s, a)$ è così definita:
 - Se a è applicabile in s :
$$\gamma(s, a) = (s \setminus effects^-(a)) \cup effects^+(a)$$
 - Altrimenti, $\gamma(s, a)$ è indefinita
-
- **OSS.** S è chiusa in γ
 - STRIPS usa la progression per modificare lo stato corrente a cui si giunge eseguendo le azioni fin qui selezionate nel piano in costruzione

Regression: Calcolo del (sotto)goal predecessore

Regression

- Il calcolo del **regresso**, ovvero il *sottogoal predecessore di un goal dato*, avviene nel seguente modo:
 - dato un goal complesso come congiunzione di goal atomici
$$g : g_1 \wedge g_2, \dots, \wedge g_n$$
 - sia a azione ground tale che $g_i \in effects^+(a)$ e $g_i \in g$
 - $g' = \gamma^{-1}(g, a) = (g \setminus effects^+(a)) \cup pre(a)$
- g' è quindi il regresso di g attraverso la relazione di transizione γ^{-1} e l'azione a
- NB. *nel calcolo del predecessore non sono considerati gli effetti negativi perché?*
- STRIPS usa la regressione per "ridurre la distanza" tra il goal che si vuole raggiungere e lo stato corrente

STRIPS: Algoritmo

STRIPS (*initState*, *goals*)

state = *initState*; *plan* = []; *stack* = []

Push *goals* on *stack*

Repeat until *stack* is empty

- **If** top of *stack* is a goal *g* satisfied in *state*, **then pop** *stack*
- **Else if** top of *stack* is a conjunctive goal *g*, then
 - Select** an ordering for the subgoals of *g*, and **push** them on *stack*
- **Else if** top of *stack* is a simple goal *sg*, then
 - Choose** an operator *o* whose *effects*⁺ matches goal *sg*
 - Replace** goal *sg* with operator *o*
 - Push** the preconditions of *o* on *stack*
- **Else if** top of *stack* is an operator *o*, then
 - state* = **apply**(*o*, *state*)
 - plan* = [*plan*; *o*]

STRIPS: Algoritmo

STRIPS (*initState*, *goals*)

state = *initState*; *plan* = []; *stack* = []

Push *goals* on *stack*

Repeat until *stack* is empty

- **If** top of *stack* is a goal *g* satisfied in *state*, **then pop** *stack*
- **Else if** top of *stack* is a conjunctive goal *g*, **then**
 Select an ordering for the subgoals of *g*, and **push** them on *stack*
- **Else if** top of *stack* is a simple goal *sg*, **then**
 Choose an operator *o* whose *effects*⁺ matches goal *sg*
 Replace goal *sg* with operator *o*
 Push the preconditions of *o* on *stack*
- **Else if** top of *stack* is an operator *o*, **then**
 state = **apply**(*o*, *state*)
 plan = [*plan*; *o*]

se in cima allo *stack* c'è un goal (atomico o complesso) già soddisfatto nello stato corrente, non ho nulla da fare.

STRIPS: Algoritmo

STRIPS (*initState*, *goals*)

state = *initState*; *plan* = []; *stack* = []

Push *goals* on *stack*

Repeat until *stack* is empty

- **If** top of *stack* is a goal *g* satisfied in *state*, **then pop** *stack*
- **Else if** top of *stack* is a conjunctive goal *g*, **then**
 Select an ordering for the subgoals of *g*, and **push** them on *stack*
- **Else if** top of *stack* is a simple goal *sg*, **then**
 Choose an operator *o* whose *effects*⁺ matches goal *sg*
 Replace goal *sg* with operator *o*
 Push the preconditions of *o* on *stack*
- **Else if** top of *stack* is an operator *o*, **then**
 state = **apply**(*o*, *state*)
 plan = [*plan*; *o*]

se in cima allo *stack* c'è un goal complesso, allora semplificalo (linearizzazione).

STRIPS: Algoritmo

STRIPS (*initState*, *goals*)

state = *initState*; *plan* = []; *stack* = []

Push *goals* on *stack*

Repeat until *stack* is empty

- **If** top of *stack* is a goal *g* satisfied in *state*, **then pop** *stack*
- **Else if** top of *stack* is a conjunctive goal *g*, **then**
 Select an ordering for the subgoals of *g*, and **push** them on *stack*
- **Else if** top of *stack* is a simple goal *sg*, **then**
 Choose an operator *o* whose *effects*⁺ matches goal *sg*
 Replace goal *sg* with operator *o*
 Push the preconditions of *o* on *stack*
- **Else if** top of *stack* is an operator *o*, **then**
 state = **apply**(*o*, *state*)
 plan = [*plan*; *o*]

se in cima allo *stack* c'è un goal atomico, allora risolvalo (means-end).

STRIPS: Algoritmo

STRIPS (*initState*, *goals*)

state = *initState*; *plan* = []; *stack* = []

Push *goals* on *stack*

Repeat until *stack* is empty

- **If** top of *stack* is a goal *g* satisfied in *state*, **then pop** *stack*
- **Else if** top of *stack* is a conjunctive goal *g*, **then**
 - Select** an ordering for the subgoals of *g*, and **push** them on *stack*
- **Elseif** top of *stack* is a simple goal *sg*, **then**
 - Choose** an operator *o* whose *effects*⁺ matches goal *sg*
 - Replace** goal *sg* with operator *o*
 - Push** the preconditions of *o* on *stack*
- **Else if top of stack is an operator *o*, then**
 - state* = **apply**(*o*, *state*)
 - plan* = [*plan*; *o*]

se in cima allo stack c'è un operatore completamente istanziato (azione), allora esegui ()modifica lo stato corrente) e aggiungilo al piano in costruzione.

Un esempio di pianificazione

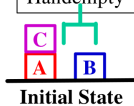
1. Stack

On(A, C) On(C, B)



State

Clear(B)
Clear(C)
On(C, A)
On(A, Table)
On(B, Table)
Handempty



Un esempio di pianificazione

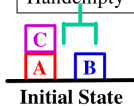
1. Stack

On(A, C) On(C, B)



State

Clear(B)
Clear(C)
On(C, A)
On(A, Table)
On(B, Table)
Handempty



2. Stack

On(A, C) On(C, B)

On(A, C)

On(C, B)

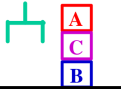
State

Clear(B)
Clear(C)
On(C, A)
On(A, Table)
On(B, Table)
Handempty

Un esempio di pianificazione

1. Stack

On(A, C) On(C, B)



Goal

State

Clear(B)
Clear(C)
On(C, A)
On(A, Table)
On(B, Table)
Handempty



Initial State

2. Stack

On(A, C) On(C, B)

On(A, C)

On(C, B)

State

Clear(B)
Clear(C)
On(C, A)
On(A, Table)
On(B, Table)
Handempty

3. Stack

On(A, C) On(C, B)

On(A, C)

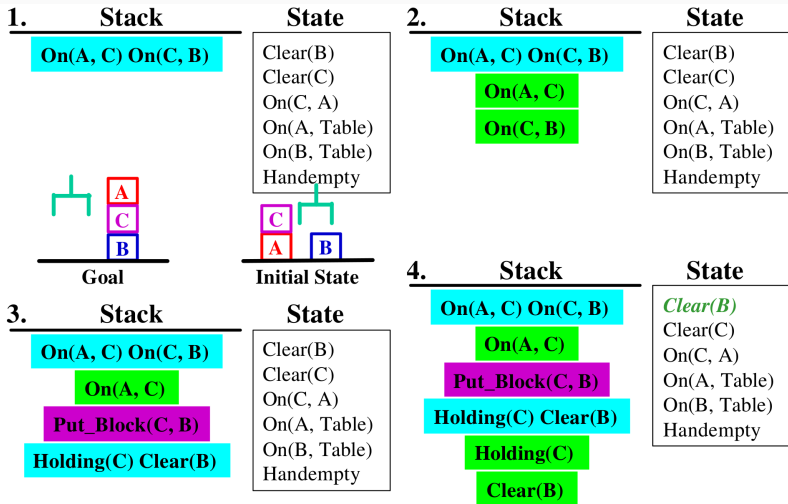
Put_Block(C, B)

Holding(C) Clear(B)

State

Clear(B)
Clear(C)
On(C, A)
On(A, Table)
On(B, Table)
Handempty

Un esempio di pianificazione



Un esempio di pianificazione

5. Stack

On(A, C) On(C, B)

On(A, C)

Put_Block(C, B)

Holding(C) Clear(B)

Holding(C)

State

Clear(B)

Clear(C)

On(C, A)

On(A, Table)

On(B, Table)

Handempty

Un esempio di pianificazione

5. Stack

On(A, C) On(C, B)

On(A, C)

Put_Block(C, B)

Holding(C) Clear(B)

Holding(C)

State

Clear(B)
Clear(C)
On(C, A)
On(A, Table)
On(B, Table)
Handempty

6. Stack

On(A, C) On(C, B)

On(A, C)

Put_Block(C, B)

Holding(C) Clear(B)

Pick_Block(C)

Handempty Clear(C) On(C, ?b)

State

Clear(B)
Clear(C)
On(C, A)
On(A, Table)
On(B, Table)
Handempty

Un esempio di pianificazione

5. Stack

On(A, C) On(C, B)

On(A, C)

Put_Block(C, B)

Holding(C) Clear(B)

Holding(C)

State

Clear(B)
Clear(C)
On(C, A)
On(A, Table)
On(B, Table)
Handempty

6. Stack

On(A, C) On(C, B)

On(A, C)

Put_Block(C, B)

Holding(C) Clear(B)

Pick_Block(C)

Handempty Clear(C) On(C, ?b)

State

Clear(B)
Clear(C)
On(C, A)
On(A, Table)
On(B, Table)
Handempty

7. Stack

On(A, C) On(C, B)

On(A, C)

Put_Block(C, B)

Holding(C) Clear(B)

Pick_Block(C)

State

Clear(B)
Clear(C)
On(C, A)
On(A, Table)
On(B, Table)
Handempty

Un esempio di pianificazione

5. Stack

On(A, C) On(C, B)

On(A, C)

Put_Block(C, B)

Holding(C) Clear(B)

Holding(C)

State

Clear(B)
Clear(C)
On(C, A)
On(A, Table)
On(B, Table)
Handempty

6. Stack

On(A, C) On(C, B)

On(A, C)

Put_Block(C, B)

Holding(C) Clear(B)

Pick_Block(C)

Handempty Clear(C) On(C, ?b)

State

Clear(B)
Clear(C)
On(C, A)
On(A, Table)
On(B, Table)
Handempty

7. Stack

On(A, C) On(C, B)

On(A, C)

Put_Block(C, B)

Holding(C) Clear(B)

Pick_Block(C)

State

Clear(B)
Clear(C)
On(C, A)
On(A, Table)
On(B, Table)
Handempty

8. Stack

On(A, C) On(C, B)

On(A, C)

Put_Block(C, B)

Holding(C) Clear(B)

State

Clear(B)
Clear(C)
On(A, Table)
On(B, Table)
Holding(C)
Clear(A)

[Pick(C)]

Un esempio di pianificazione

9. Stack

On(A, C) On(C, B)

On(A, C)

Put_Block(C, B)

[Pick(C)]

State

Clear(B)

Clear(C)

On(A, Table)

On(B, Table)

Holding(C)

Clear(A)

Un esempio di pianificazione

9. Stack

On(A, C) On(C, B)

On(A, C)

Put_Block(C, B)

[Pick(C)]

State

Clear(B)

Clear(C)

On(A, Table)

On(B, Table)

Holding(C)

Clear(A)

10. Stack

On(A, C) On(C, B)

On(A, C)

[Pick(C); Put(C, B)]

State

Clear(C)

On(A, Table)

On(B, Table)

Clear(A)

Handempty

On(C, B)

Un esempio di pianificazione

9. Stack

On(A, C) On(C, B)

On(A, C)

Put_Block(C, B)

State

Clear(B)

Clear(C)

On(A, Table)

On(B, Table)

Holding(C)

Clear(A)

[Pick(C)]

10. Stack

On(A, C) On(C, B)

On(A, C)

State

Clear(C)

On(A, Table)

On(B, Table)

Clear(A)

Handempty

On(C, B)

[Pick(C); Put(C, B)]

11. Stack

On(A, C) On(C, B)

Put_Block(A, C)

Holding(A) Clear(C)

State

Clear(C)

On(A, Table)

On(B, Table)

Clear(A)

Handempty

On(C, B)

[Pick(C); Put(C, B)]

Un esempio di pianificazione

9. Stack

On(A, C) On(C, B)

On(A, C)

Put_Block(C, B)

State

Clear(B)

Clear(C)

On(A, Table)

On(B, Table)

Holding(C)

Clear(A)

[Pick(C)]

10. Stack

On(A, C) On(C, B)

On(A, C)

State

Clear(C)

On(A, Table)

On(B, Table)

Clear(A)

Handempty

On(C, B)

[Pick(C); Put(C, B)]

11. Stack

On(A, C) On(C, B)

Put_Block(A, C)

Holding(A) Clear(C)

State

Clear(C)

On(A, Table)

On(B, Table)

Clear(A)

Handempty

On(C, B)

[Pick(C); Put(C, B)]

12. Stack

On(A, C) On(C, B)

Put_Block(A, C)

Holding(A) Clear(C)

Holding(A)

Clear(C)

State

Clear(C)

On(A, Table)

On(B, Table)

Clear(A)

Handempty

On(C, B)

[Pick(C); Put(C, B)]

Un esempio di pianificazione

13.	Stack	State
	On(A, C) On(C, B)	Clear(C)
	Put_Block(A, C)	On(A, Table)
	Holding(A) Clear(C)	On(B, Table)
	Holding(A)	Clear(A)
		Handempty
		On(C, B)

[Pick(C); Put(C, B)]

Un esempio di pianificazione

13. Stack

On(A, C) On(C, B)

Put_Block(A, C)

Holding(A) Clear(C)

Holding(A)

State

Clear(C)
On(A, Table)
On(B, Table)
Clear(A)
Handempty
On(C, B)

[Pick(C); Put(C, B)]

14. Stack

On(A, C) On(C, B)

Put_Block(A, C)

Holding(A) Clear(C)

Pick_Table(A)

Handempty Clear(A)
On(A, Table)

State

Clear(C)
On(A, Table)
On(B, Table)
Clear(A)
Handempty
On(C, B)

[Pick(C); Put(C, B)]

Un esempio di pianificazione

13. Stack

On(A, C) On(C, B)

Put_Block(A, C)

Holding(A) Clear(C)

Holding(A)

State

Clear(C)
On(A, Table)
On(B, Table)
Clear(A)
Handempty
On(C, B)

[Pick(C); Put(C, B)]

15. Stack

On(A, C) On(C, B)

Put_Block(A, C)

Holding(A) Clear(C)

Pick_Table(A)

State

Clear(C)
On(A, Tab
On(B, Tab.
Clear(A)
Handempt
On(C, B)

[Pick(C); Put(C, B)]

14. Stack

On(A, C) On(C, B)

Put_Block(A, C)

Holding(A) Clear(C)

Pick_Table(A)

Handempty Clear(A)
On(A, Table)

State

Clear(C)
On(A, Table)
On(B, Table)
Clear(A)
Handempty
On(C, B)

[Pick(C); Put(C, B)]

Un esempio di pianificazione

13. Stack

On(A, C) On(C, B)

Put_Block(A, C)

Holding(A) Clear(C)

Holding(A)

State

Clear(C)
On(A, Table)
On(B, Table)
Clear(A)
Handempty
On(C, B)

[Pick(C); Put(C, B)]

15. Stack

On(A, C) On(C, B)

Put_Block(A, C)

Holding(A) Clear(C)

Pick_Table(A)

State

Clear(C)
On(A, Table)
On(B, Table)
Clear(A)
Handempty
On(C, B)

[Pick(C); Put(C, B)]

14. Stack

On(A, C) On(C, B)

Put_Block(A, C)

Holding(A) Clear(C)

Pick_Table(A)

Handempty Clear(A)
On(A, Table)

State

Clear(C)
On(A, Table)
On(B, Table)
Clear(A)
Handempty
On(C, B)

[Pick(C); Put(C, B)]

16. Stack

On(A, C) On(C, B)

Put_Block(A, C)

Holding(A) Clear(C)

State

Clear(C)
On(B, Table)
Clear(A)
On(C, B)
Holding(A)

[Pick(C); Put(C, B); PickT(A)]

17. Stack

On(A, C) On(C, B)

Put_Block(A, C)

State

Clear(C)

On(B, Table)

Clear(A)

On(C, B)

Holding(A)

[Pick(C); Put(C, B); PickT(A)]

Un esempio di pianificazione

17. Stack

On(A, C) On(C, B)

Put_Block(A, C)

State

Clear(C)

On(B, Table)

Clear(A)

On(C, B)

Holding(A)

[Pick(C); Put(C, B); PickT(A)]

18. Stack

On(A, C) On(C, B)

State

On(B, Table)

Clear(A)

On(C, B)

Handempty

On(A, C)

[Pick(C); Put(C, B); PickT(A); Put(A, C)]

Un esempio di pianificazione

17. Stack

On(A, C) On(C, B)

Put_Block(A, C)

State

Clear(C)
On(B, Table)
Clear(A)
On(C, B)
Holding(A)

[Pick(C); Put(C, B); PickT(A)]

18. Stack

On(A, C) On(C, B)

State

On(B, Table)
Clear(A)
On(C, B)
Handempty
On(A, C)

[Pick(C); Put(C, B); PickT(A); Put(A, C)]

19. Stack

State

On(B, Table)
Clear(A)
On(C, B)
Handempty
On(A, C)

[Pick(C); Put(C, B); PickT(A); Put(A, C)]

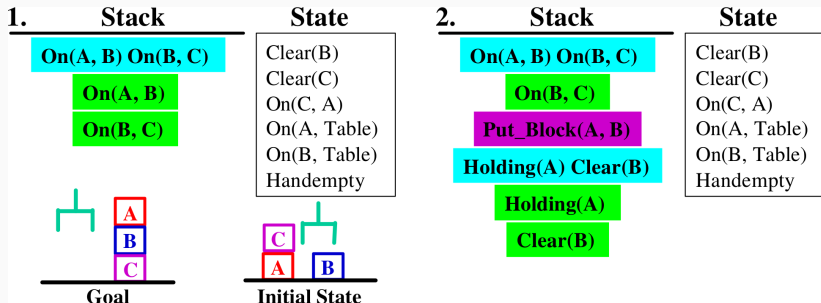
- Implementa una forma di **linear planning**
- **Vantaggi:**
 - spazio di ricerca ridotto perché i goal sono considerati uno per volta
 - ideale quando i goal sono indipendenti tra loro
 - è *sound*
- **Svantaggi:**
 - linear planning può produrre **piani subottimi** (se consideriamo la lunghezza il parametro di qualità)
 - **linear planning è incompleto**

E' una conseguenza di:

- **interdipendenza tra i sottogoal**
- **ordinamento sfavorevole dei goal**

La combinazione di questi due elementi può dare origine ad una particolare situazione nota come **Sussmann's Anomaly**: quando cioè è necessario disfare parte dei goal già raggiunti per poter risolvere l'intero problema.

Sussmann's Anomaly: un Esempio



Sussmann's Anomaly: un Esempio

3. Stack

On(A, B) On(B, C)

On(B, C)

Put_Block(A, B)

Holding(A) Clear(B)

Holding(A)

Pick_Table(A)

Handempty Clear(A) On(A, Table)

Clear(A)

Handempty

On(A, Table)

State

Clear(B)
Clear(C)
On(C, A)
On(A, Table)
On(B, Table)
Handempty

4. Stack

On(A, B) On(B, C)

On(B, C)

Put_Block(A, B)

Holding(A) Clear(B)

Holding(A)

Pick_Table(A)

Handempty Clear(A) On(A, Table)

Pick_Block(C, A)

Handempty Clear(C) On(C, A)

State

Clear(B)
Clear(C)
On(C, A)
On(A, Table)
On(B, Table)
Handempty

Sussmann's Anomaly: un Esempio

5. Stack State

On(A, B) On(B, C)

On(B, C)

Put_Block(A, B)

Holding(A) Clear(B)

Holding(A)

Pick_Table(A)

Handempty Clear(A) On(A, Table)

[Pick(C,A)]

Clear(B)
Clear(C)
On(A, Table)
On(B, Table)
Holding(C)

6. Stack State

On(A, B) On(B, C)

On(B, C)

Put_Block(A, B)

Holding(A) Clear(B)

Holding(A)

Pick_Table(A)

Handempty Clear(A) On(A, Table)

Put_Table(C)

Holding(C)

[Pick(C,A)]

Clear(B)
Clear(C)
On(A, Table)
On(B, Table)
Holding(C)

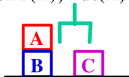
Sussmann's Anomaly: un Esempio

7. Stack

On(A, B) On(B, C)

On(B, C)

[Pick(C,A); PutT(C);
PickT(A); Put(A, B)]



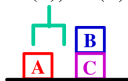
State

Clear(C)
On(B, Table)
On(C, Table)
Clear(A)
On(A, B)
Handempty

8. Stack

On(A, B) On(B, C)

[Pick(C,A); PutT(C);
PickT(A); Put(A, B);
Pick(A, B); PutT(A);
PickT(B); Put(B, C)]



State

On(C, Table)
Clear(B)
Clear(A)
On(A, Table)
On(B, C)
Handempty

9. Stack

On(A, B) On(B, C)

On(A, B)

On(B, C)

[Pick(C,A); PutT(C);
PickT(A); Put(A, B);
Pick(A, B); PutT(A);
PickT(B); Put(B, C)]

State

On(C, Table)
Clear(B)
Clear(A)
On(A, Table)
On(B, C)
Handempty

10. Stack

On(A, B) On(B, C)

[Pick(C,A); PutT(C);
PickT(A); Put(A, B);
Pick(A, B); PutT(A);
PickT(B); Put(B, C);
PickT(A); Put(A, B)]

State

On(C, Table)
Clear(A)
On(B, C)
On(A, B)
Handempty

Consideriamo questo scenario della logistica

Load(o, p, loc)

Pre: At(o, loc), At(p, loc)

Eff: Inside(o, p), $\neg$ At(o, loc)

Unload(o, p, loc)

Pre: Inside(o, p), At(p, loc)

Eff: At(o, loc), $\neg$ Inside(o, p)

Fly(p, from, to)

Pre: At(p, from), HaveFuel(p)

Eff: At(p, to), $\neg$ At(p, from), $\neg$ HaveFuel(p)

Initial state: At(Obj1, LocA), At(Obj2, LocA), At(747, LocA), HaveFuel(747)

Goal state: At(Obj1, LocB), At(Obj2, LocB)

Incompletezza

E' una conseguenza della **non reversabilità** delle azioni in particolari domini.

Ad esempio, nel caso della logistica ...

Load(o, p, loc)

Pre: At(o, loc), At(p, loc)

Eff: Inside(o, p), $\neg$ At(o, loc)

Unload(o, p, loc)

Pre: Inside(o, p), At(p, loc)

Eff: At(o, loc), $\neg$ Inside(o, p)

Fly(p, from, to)

Pre: At(p, from), **HaveFuel**(p)

Eff: At(p, to), $\neg$ At(p, from), $\neg$ **HaveFuel**(p)

Initial state: At(Obj1, LocA), At(Obj2, LocA), At(747, LocA), HaveFuel(747)

Goal state: At(Obj1, LocB), At(Obj2, LocB)

Incompletezza

E' una conseguenza della **non reversabilità** delle azioni in particolari domini.

Ad esempio, nel caso della logistica ...

Load(o, p, loc)

Pre: At(o, loc), At(p, loc)

Eff: Inside(o, p), $\neg$ At(o, loc)

Unload(o, p, loc)

Pre: Inside(o, p), At(p, loc)

Eff: At(o, loc), $\neg$ Inside(o, p)

Fly(p, from, to)

Pre: At(p, from), **HaveFuel**(p)

Eff: At(p, to), $\neg$ At(p, from), $\neg$ **HaveFuel**(p)

Initial state: At(Obj1, LocA), At(Obj2, LocA), At(747, LocA), HaveFuel(747)

Goal state: At(Obj1, LocB), At(Obj2, LocB)

Risolvendo linearmente:

Load(Obj1, 747, LocA), Fly(LocA, LocB), Unload(Obj1, 747, LocB)

Incompletezza

E' una conseguenza della **non reversabilità** delle azioni in particolari domini.

Ad esempio, nel caso della logistica ...

Load(o, p, loc)

Pre: At(o, loc), At(p, loc)

Eff: Inside(o, p), $\neg$ At(o, loc)

Unload(o, p, loc)

Pre: Inside(o, p), At(p, loc)

Eff: At(o, loc), $\neg$ Inside(o, p)

Fly(p, from, to)

Pre: At(p, from), **HaveFuel(p)**

Eff: At(p, to), $\neg$ At(p, from), $\neg$ **HaveFuel(p)**

Initial state: At(Obj1, LocA), At(Obj2, LocA), At(747, LocA), HaveFuel(747)

Goal state: At(Obj1, LocB), At(Obj2, LocB)

Risolvendo linearmente:

Load(Obj1, 747, LocA), Fly(LocA, LocB), Unload(Obj1, 747, LocB)

Ma adesso non abbiamo più carburante per tornare indietro!!

PDDL Planning Domain Definition Language

- PDDL 1.0 $\equiv$ STRIPS language
- Standard di fatto per definire l'input dei pianificatori
- Usato nelle competizioni internazionali

Il mondo dei blocchi in PDDL semplificato

Init($On(A, Table) \wedge On(B, Table) \wedge On(C, A)$
 $\wedge Block(A) \wedge Block(B) \wedge Block(C) \wedge Clear(B) \wedge Clear(C)$)
Goal($On(A, B) \wedge On(B, C)$)
Action(*Move*(b, x, y),
 PRECOND: $On(b, x) \wedge Clear(b) \wedge Clear(y) \wedge Block(b) \wedge Block(y) \wedge$
 $(b \neq x) \wedge (b \neq y) \wedge (x \neq y)$,
 EFFECT: $On(b, y) \wedge Clear(x) \wedge \neg On(b, x) \wedge \neg Clear(y)$)
Action(*MoveToTable*(b, x),
 PRECOND: $On(b, x) \wedge Clear(b) \wedge Block(b) \wedge (b \neq x)$,
 EFFECT: $On(b, Table) \wedge Clear(x) \wedge \neg On(b, x)$)

Figure 10.3 A planning problem in the blocks world: building a three-block tower. One solution is the sequence [*MoveToTable*(C, A), *Move*($B, Table, C$), *Move*($A, Table, B$)].

Consideriamo lo schema d'azione $\text{Move}(b, x, y)$

Problema 1: potere espressivo: blocchi liberi come preconditione della Move

- In FOL $\forall x \neg \text{On}(x, b)$ oppure $\neg \exists \text{On}(x, b)$
- In PDDL si aggiunge un predicato che rappresenta esplicitamente questa situazione: $\text{Clear}(b)$

Consideriamo lo schema d'azione $\text{Move}(b, x, y)$

Problema 2: proposizionalizzazione con l'oggetto Table

- quando x o y è Table il predicato $\text{Clear}(\text{Table})$ non ha molto senso:
 - se $x=\text{Table}$ l'effetto è che il tavolo diventi "*clear*", ma il tavolo non dovrebbe svuotarsi perché c'è almeno un altro blocco
 - se $y=\text{Table}$ la preconditione è che il tavolo sia libero, ma di nuovo il tavolo non può essere libero nel senso di sgombro da blocchi, perché c'è almeno un altro blocco
- Soluzione: introdurre un nuovo schema di azione (plan operator) che usi il tavolo come parametro implicito: $\text{MoveToTable}(b, x)$

Init($At(C_1, SFO) \wedge At(C_2, JFK) \wedge At(P_1, SFO) \wedge At(P_2, JFK)$
 $\wedge Cargo(C_1) \wedge Cargo(C_2) \wedge Plane(P_1) \wedge Plane(P_2)$
 $\wedge Airport(JFK) \wedge Airport(SFO)$)
Goal($At(C_1, JFK) \wedge At(C_2, SFO)$)
Action(*Load*(c, p, a),
 PRECOND: $At(c, a) \wedge At(p, a) \wedge Cargo(c) \wedge Plane(p) \wedge Airport(a)$
 EFFECT: $\neg At(c, a) \wedge In(c, p)$)
Action(*Unload*(c, p, a),
 PRECOND: $In(c, p) \wedge At(p, a) \wedge Cargo(c) \wedge Plane(p) \wedge Airport(a)$
 EFFECT: $At(c, a) \wedge \neg In(c, p)$)
Action(*Fly*($p, from, to$),
 PRECOND: $At(p, from) \wedge Plane(p) \wedge Airport(from) \wedge Airport(to)$
 EFFECT: $\neg At(p, from) \wedge At(p, to)$)

Figure 10.1 A PDDL description of an air cargo transportation planning problem.

Problemi di modellazione

Problema 1: potere espressivo: posizione delle merci

- Vogliamo dire che quando un aereo si sposta da un aeroporto all'altro, tutte le merci che sono al suo interno si spostano con esso.
- In FOL sarebbe semplice imporre questa condizione utilizzando il quantificatore universale
- PDDL è sprovvisto di quantificatori
- Soluzione: Uso di due predicati per rappresentare la posizione di una merce:
 - $In(c, p)$ merci c a bordo aereo p
 - $At(x, a)$ oggetto x (aereo o merce) in aeroporto a

Problema 2: proposizionalizzazione

- La proposizionalizzazione degli operatori potrebbe produrre azioni spurie: es $Fly(AF99, JFK, JFK)$
- Soluzioni possibili:
 - Ignorare il problema ma...
 - Imporre un vincolo di disequivalenza tra *from* e *to* nella definizione dello schema *Fly*

Esempio di plan operator in PDDL

```
(: action MOVE
: parameters(?block BLOCK ?from OBJECT
               ?to OBJECT)
: preconds(and (clear ?block)
              (clear ?to)
              (on ?block ?from))
: effects(and
           (on ?bloc ?to)
           (not (on ?block ?from)) )
```

- parametri sono variabili
tipate (gerarchia di tipi)

Esempio di plan operator in PDDL

```
(: action MOVE
: parameters (?block BLOCK ?from OBJECT
               ?to OBJECT)
: preconds(and (clear ?block)
               (clear ?to)
               (on ?block ?from))
: effects(and
          (on ?block ?to)
          (not (on ?block ?from))) )
```

- parametri sono variabili tipate (gerarchia di tipi)
- precondizioni come congiunzione di predicati non istanziati

Esempio di plan operator in PDDL

```
(:action MOVE
:parameters(?block BLOCK ?from OBJECT
            ?to OBJECT)
:preconds(and (clear ?block)
             (clear ?to)
             (on ?block ?from))
:effects(and
         (on ?block ?to)
         (not (on ?block ?from)) )
```

- parametri sono variabili tipate (gerarchia di tipi)
- precondizioni come congiunzione di predicati non istanziati
- effetti come due liste add-list e delete-list (preceduta dal not)

- **Conditional Effects:** Stabilire che alcuni effetti sono raggiunti solo se sono vere certe condizioni

```
(when <condition>  
  <effect>)
```

- **Durative Actions:** Un plan operator si divide in tre parti:
 - un evento iniziale
 - un evento finale
 - una condizione invariante che deve permanere per tutta la durata dell'azione
- **Numeric Fluent:** Es. consentono di tracciare il consumo di risorse nel tempo
- **Quantifiers:**

```
(forall (?v1 ?v2 ...) condition_formula)  
(exists (?v1 ?v2 ...) condition_formula)
```